

AI Memory Bus

A Shared Intelligence Layer for Local AI Applications

Document Information

Version	1.0.0
Date	May 21, 2026
Audience	Beginners & Developers
Status	Complete Reference

Contents

1	Introduction	4
1.1	What Is the AI Memory Bus?	4
1.1.1	A Human Analogy	4
1.2	Who Is This For?	4
1.3	Core Properties	5
2	The Problem Being Solved	6
2.1	The AI Amnesia Problem	6
2.1.1	What Users Experience Today	6
2.1.2	Estimated Time Wasted per Year	6
2.2	Current Landscape vs. AI Memory Bus	7
3	System Architecture	8
3.1	High-Level Overview	8
3.2	Internal Component Breakdown	8
4	The Four Memory Types	10
4.1	Episodic Memory — <i>What Happened and When</i>	10
4.2	Semantic Memory — <i>What Is True</i>	11
4.3	Procedural Memory — <i>How Things Get Done</i>	11
4.4	Working Memory — <i>What Is Happening Right Now</i>	12
4.5	Memory Type Comparison Table	12
5	Data Structures and Algorithms	13
5.1	Overview	13
5.2	HashMap (<code>dict</code>) — Working Memory Core	13
5.3	Min-Heap (<code>heapq</code>) — TTL Expiry	14
5.4	OrderedDict — LRU Eviction	14

5.5	HNSW Graph — Semantic Vector Search	15
6	Vector Embeddings and Semantic Search	16
6.1	What Is a Vector Embedding?	16
6.2	The Embedding Pipeline	16
6.3	Cosine Similarity — The Mathematics	16
6.4	Why This Matters — Keyword vs. Semantic Search	17
7	Memory Decay and Confidence Scoring	18
7.1	Why Memories Need to Decay	18
7.2	The Confidence Score Formula	18
7.3	Decay Curve	19
8	API Reference	20
8.1	Health Check	20
8.2	Write a Memory	20
8.3	Recall by Semantic Search	20
8.4	Recall by Time Range (Episodic)	21
8.5	Working Memory Operations	21
8.6	Full Endpoint Reference	22
9	Workflow Diagrams	23
9.1	Write Flow	23
9.2	Recall Flow	23
9.3	Cross-Application Memory Sharing	23
10	Use Cases	25
10.1	Use Case 1: Instant Onboarding for New AI Tools	25
10.2	Use Case 2: Job Deduplication	25
10.3	Use Case 3: Cross-Tool Active Context	26
10.4	Use Case 4: Developer History Recall	26
10.5	Use Case Summary	27
11	Technology Stack	28
11.1	Dependencies	28
12	Implementation Roadmap	29

12.1 Phase-by-Phase Plan	29
12.2 Estimated Timeline	29
13 Future Scope	31
14 Conclusion	32

Chapter 1

Introduction

1.1 What Is the AI Memory Bus?

The **AI Memory Bus** is a persistent, local background service that acts as a shared memory layer for every AI-powered application running on a single computer. Think of it as the *hippocampus of your machine* — the region of the brain responsible for forming and retrieving memories, implemented as software.

Simple One-Line Definition

AI Memory Bus = A local server that stores what AI tools learn about you, so every AI application on your PC shares the same knowledge — permanently, privately, without you repeating yourself.

1.1.1 A Human Analogy

Imagine you have five assistants at work. Every morning each one forgets everything from the previous day. You must re-introduce yourself, re-explain your project, re-state your preferences — every single day, to every single assistant.

This is exactly what happens with AI tools today. The AI Memory Bus gives all your assistants a shared, permanent notebook. What one assistant learns, all others can immediately access.

1.2 Who Is This For?

- **Developers** who use multiple AI tools simultaneously (Copilot, Claude, Ollama, custom scripts)
- **Power users** who want AI tools to know them deeply without constant re-configuration
- **Teams** who want shared context across developers and AI tools
- **Builders** creating AI applications that need persistent memory

1.3 Core Properties

Property	Description
Local-first	All data stays on your PC. Nothing sent to the cloud.
Privacy-first	You own your memory. Can be encrypted at rest.
Cross-app	Any application on the same machine can read and write.
Persistent	Survives reboots, updates, and new tool installations.
Semantic	Understands <i>meaning</i> , not just keywords.
Multi-typed	Stores different kinds of knowledge in different ways.

Chapter 2

The Problem Being Solved

2.1 The AI Amnesia Problem

Every AI application — GitHub Copilot, Claude Desktop, local LLMs running via Ollama, custom Python scripts — starts each session with zero knowledge of the user. This is by design for privacy reasons in cloud tools, but it creates a deeply frustrating experience.

2.1.1 What Users Experience Today

```
1 # Monday -- talking to GitHub Copilot
2 User: "I prefer functional programming, use Python, work at Aurum Data
   Solutions"
3 Copilot: "Got it!" # <-- forgets by tomorrow
4
5 # Tuesday -- talking to Claude Desktop
6 User: "I prefer functional programming, use Python, work at Aurum Data
   Solutions"
7 Claude: "Understood!" # <-- forgets next session
8
9 # Wednesday -- brand new AI tool installed
10 User: # has to explain everything from scratch again
```

Listing 2.1: The repetition problem in practice

2.1.2 Estimated Time Wasted per Year

Fact Type	Re-explained / Year	Minutes Wasted
Tech stack and preferences	$\approx 200\times$	≈ 400 min
Project context	$\approx 300\times$	≈ 900 min
Coding style	$\approx 150\times$	≈ 300 min
Team and company context	$\approx 100\times$	≈ 200 min
Total	$\approx 750\times$	≈ 30 hours/year

2.2 Current Landscape vs. AI Memory Bus

Feature	Current AI Tools	AI Memory Bus
Memory scope	Single app only	All apps on the PC
Memory duration	Single session	Permanent
Storage location	Cloud (privacy risk)	Local machine
Memory types	Flat conversation log	4 structured types
Search method	Keyword only	Semantic (meaning)
Cross-app sharing	None	Full
User control	Limited	Complete

Chapter 3

System Architecture

3.1 High-Level Overview

The AI Memory Bus follows a **hub-and-spoke architecture**. The Memory Bus is the central hub; all AI applications are the spokes. Communication happens over a local HTTP REST API on `localhost:6400`.

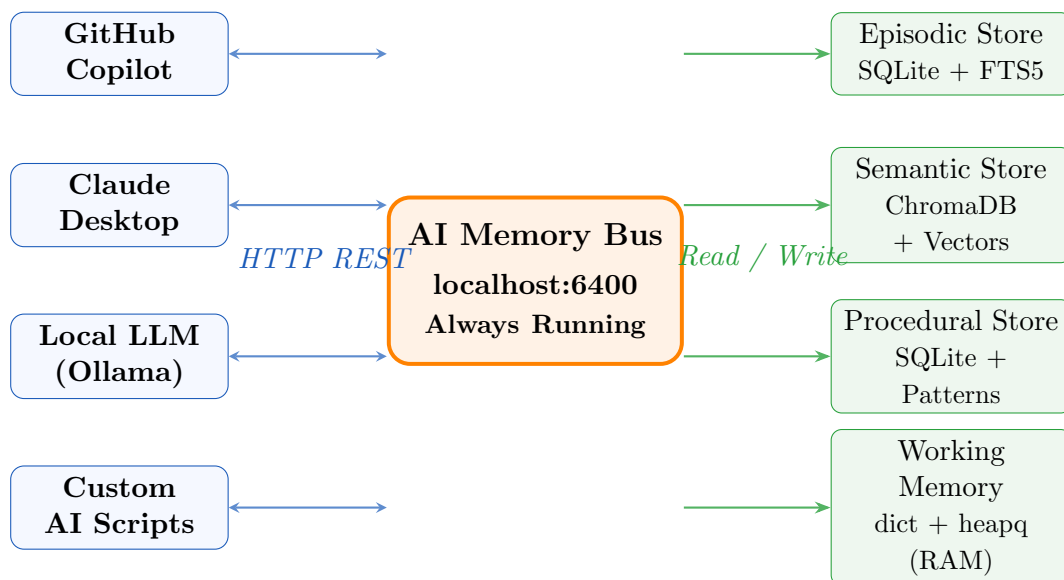


Figure 3.1: AI Memory Bus: hub-and-spoke architecture

3.2 Internal Component Breakdown

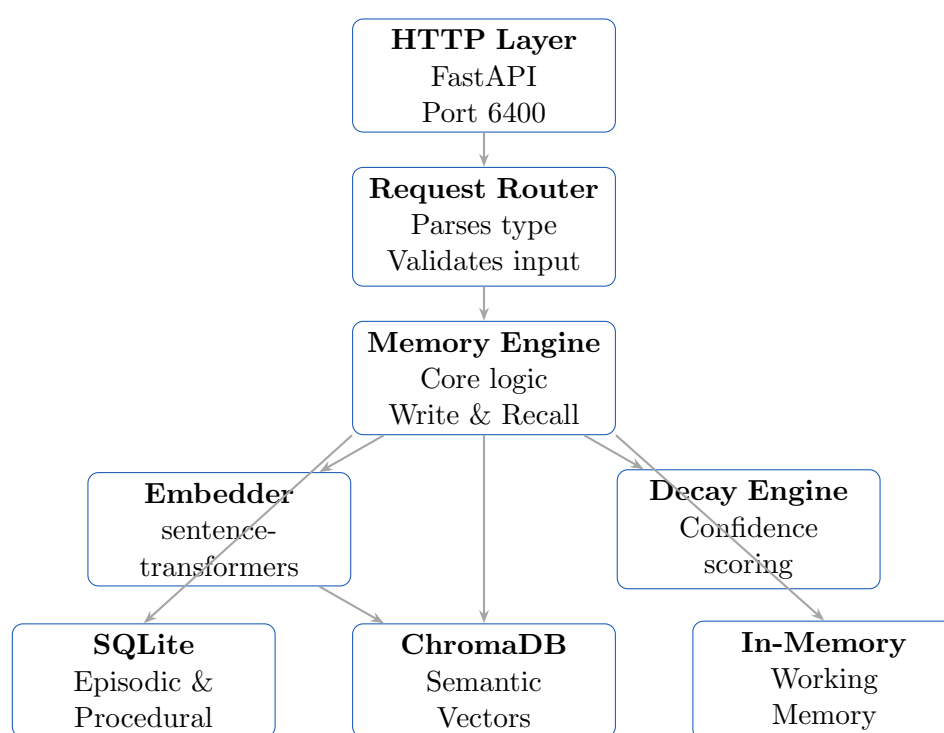


Figure 3.2: Internal component architecture of the Memory Bus

Chapter 4

The Four Memory Types

The AI Memory Bus is modelled after human memory systems from cognitive science. There are four distinct memory types, each optimised for a different kind of knowledge.

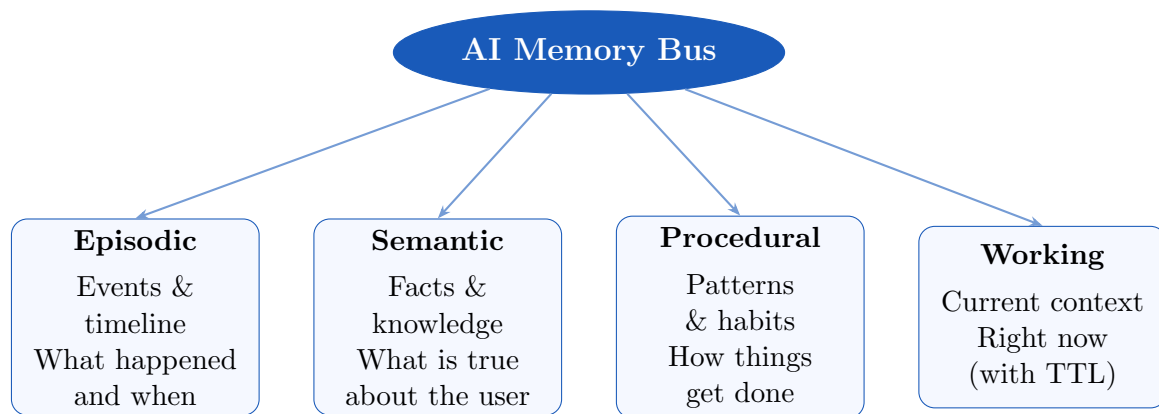


Figure 4.1: The four memory types of the AI Memory Bus

4.1 Episodic Memory — *What Happened and When*

Definition

Episodic memory stores **time-stamped events and experiences**. It answers: *“What did the user do, and when did they do it?”*

```
1 [2026-05-19 14:32] type=episodic
2   content: "Debugged auth token expiry bug in user-service"
3   tags: [debugging, auth, user-service]
4
5 [2026-05-20 09:15] type=episodic
6   content: "Refactored payment module to async/await"
7   tags: [refactor, payment, async]
8
9 [2026-05-21 11:00] type=episodic
10  content: "Design discussion about local caching strategy"
11  tags: [design, caching, architecture]
```

Listing 4.1: Sample episodic memory entries

Storage: SQLite with FTS5 full-text search extension, indexed on `timestamp` for $O(\log n)$ time-range queries.

4.2 Semantic Memory — *What Is True*

Definition

Semantic memory stores **persistent facts and knowledge** about the user. It answers: *“What is true about this person and their world?”*

```
1 { type: "semantic", category: "preference",  
2   content: "User prefers Python over JavaScript for backend",  
3   confidence: 0.95, reinforcements: 7 }  
4  
5 { type: "semantic", category: "context",  
6   content: "User works at Aurum Data Solutions",  
7   confidence: 0.99, reinforcements: 12 }  
8  
9 { type: "semantic", category: "stack",  
10  content: "Primary stack: Python, FastAPI, PostgreSQL, AWS",  
11  confidence: 0.92, reinforcements: 5 }
```

Listing 4.2: Sample semantic memory entries

Storage: ChromaDB (local vector database). Text is converted to 384-dimensional vectors; recall uses cosine similarity search via HNSW index.

4.3 Procedural Memory — *How Things Get Done*

Definition

Procedural memory stores **workflows, patterns, and habits** observed from the user’s behaviour. It answers: *“How does this user do things?”*

```
1 { type: "procedural", name: "pre-commit-routine",  
2   content: "Before committing: run pytest, black formatter, then mypy",  
3   confidence: 0.93, observed_count: 45 }  
4  
5 { type: "procedural", name: "deployment-flow",  
6   content: "Deploy: build Docker -> push ECR -> update ECS -> health  
   check",  
7   confidence: 0.97, observed_count: 23 }
```

Listing 4.3: Sample procedural memory entries

Storage: SQLite table with pattern tags and confidence scores.

4.4 Working Memory — *What Is Happening Right Now*

Definition

Working memory holds **short-term active context** about the current session. All entries carry a TTL (Time-To-Live) and expire automatically.

```

1 { key: "current_task",    value: "Building local cache server",  ttl:
   14400 }
2 { key: "active_file",    value: "cache_server.py",                ttl:
   1800  }
3 { key: "active_branch",  value: "feature/cache-ttl-eviction",  ttl:
   28800 }
4 { key: "recent_error",   value: "ConnectionRefusedError:6380",  ttl:
   900   }
```

Listing 4.4: Working memory – live context with TTL

Storage: Entirely in RAM using a `dict` (HashMap) for $O(1)$ lookups and a `heapq` (Min-Heap) for $O(\log n)$ TTL eviction.

4.5 Memory Type Comparison Table

Type	Persists	Search	Storage	TTL	Example Query
Episodic	Permanent	Time / FTS	SQLite	No	What did I work on Tuesday?
Semantic	Permanent	Semantic vec	ChromaDB	No	What language do I prefer?
Procedural	Permanent	Pattern / FTS	SQLite	No	How do I deploy?
Working	Session	Key lookup	RAM	Yes	What am I doing now?

Chapter 5

Data Structures and Algorithms

5.1 Overview

Six core data structures power the AI Memory Bus. Each is chosen for a specific performance characteristic.

Data Structure	Used For	Get	Insert
dict (HashMap)	Working memory store	$O(1)$	$O(1)$
OrderedDict	LRU eviction tracking	$O(1)$	$O(1)$
heapq (Min-Heap)	TTL expiry management	$O(\log n)$	$O(\log n)$
deque	Queue / list structures	$O(1)$ ends	$O(1)$ ends
HNSW Graph	Vector similarity index	$O(\log n)$	$O(\log n)$
SQLite B-tree	Episodic/Procedural store	$O(\log n)$	$O(\log n)$

5.2 HashMap (dict) — Working Memory Core

A Python dict is a hash table. It stores key-value pairs with near-constant time access regardless of how many entries it holds.

```
1 class WorkingMemory:
2     def __init__(self):
3         self._store = {}      # HashMap: key -> (value, expire_at)
4         self._ttl_heap = []   # Min-Heap: (expire_at, key)
5
6     def set(self, key: str, value, ttl: int):
7         expire_at = time.time() + ttl
8         self._store[key] = (value, expire_at)      # O(1)
9         heapq.heappush(self._ttl_heap, (expire_at, key))  # O(log n)
10
11     def get(self, key: str):
12         if key not in self._store:      # O(1)
```

```

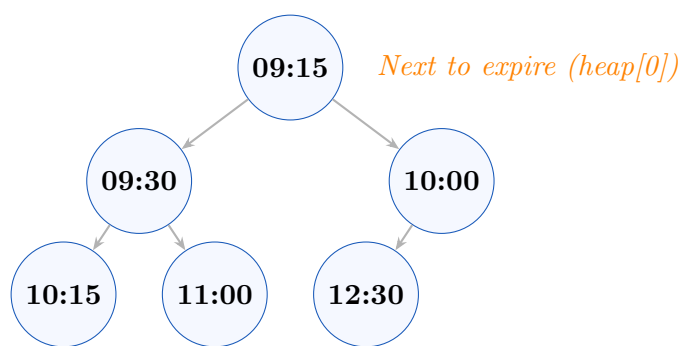
13     return None
14     value, expire_at = self._store[key]
15     if time.time() > expire_at:
16         del self._store[key]
17         return None
18     return value

```

Listing 5.1: Working memory backed by a HashMap

5.3 Min-Heap (heapq) — TTL Expiry

A min-heap is a binary tree where the smallest element is always at the root. For TTL management, entries are stored as `(expire_at, key)` tuples. The next entry to expire is always at `heap[0]`, making the check $O(1)$. Removal is $O(\log n)$.



expire_at timestamps

Figure 5.1: Min-Heap: the soonest-expiring key is always at the top

```

1 import heapq, time
2
3 def evict_expired(store: dict, heap: list):
4     now = time.time()
5     while heap and heap[0][0] <= now:           # O(1) to peek
6         expire_at, key = heapq.heappop(heap)    # O(log n) to remove
7         if key in store and store[key][1] == expire_at:
8             del store[key]

```

Listing 5.2: TTL expiry using min-heap

5.4 OrderedDict — LRU Eviction

When working memory reaches capacity, the **Least Recently Used** (LRU) entry is evicted. Python's `OrderedDict` tracks insertion/access order and allows moving items to the end in $O(1)$.

```

1 from collections import OrderedDict
2
3 class LRUWorkingMemory:
4     def __init__(self, max_size=500):

```

```

5     self._lru = OrderedDict()
6     self._max = max_size
7
8     def touch(self, key):
9         self._lru.move_to_end(key)          # O(1) -- mark as recently used
10
11    def evict_if_full(self):
12        if len(self._lru) >= self._max:
13            oldest, _ = self._lru.popitem(last=False)  # O(1) -- remove
14            LRU
15            return oldest

```

Listing 5.3: LRU eviction with OrderedDict

5.5 HNSW Graph — Semantic Vector Search

The **Hierarchical Navigable Small World** (HNSW) graph organises vector embeddings in a layered structure, enabling approximate nearest-neighbour (ANN) search in $O(\log n)$ — far faster than brute-force $O(n)$ linear scan.

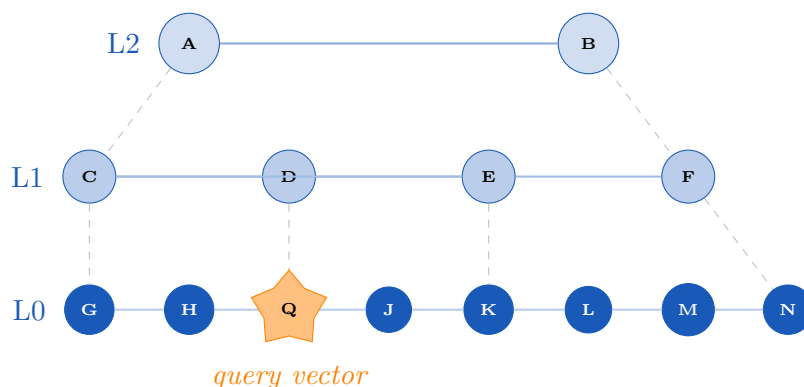


Figure 5.2: HNSW: search starts at the sparse top layer, narrows down to the exact neighbourhood at layer 0

Chapter 6

Vector Embeddings and Semantic Search

6.1 What Is a Vector Embedding?

A vector embedding is a **list of numbers** that represents the *meaning* of a piece of text. Similar meanings produce similar numbers, allowing us to find related memories even when the words are entirely different.

Core Concept

Text is converted into a point in 384-dimensional space. Memories that mean similar things cluster near each other. Finding related memories = finding the nearest points in that space.

6.2 The Embedding Pipeline

384 numbers;

[arr] (t1)-(t2); [arr] (t2)-(t3); [arr] (t3)-(t4);

6.3 Cosine Similarity — The Mathematics

To measure how similar two memories are, we compute the **cosine similarity** between their embedding vectors:

$$\text{similarity}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \cdot \sqrt{\sum_{i=1}^n B_i^2}} \quad (6.1)$$

- $\mathbf{A}, \mathbf{B}$ — the 384-dimensional embedding vectors
- Result range: $[-1, +1]$ where $+1$ means identical meaning

- Values ≥ 0.75 are typically considered highly relevant matches

```
1 import numpy as np
2
3 def cosine_similarity(vec_a: list, vec_b: list) -> float:
4     a = np.array(vec_a)
5     b = np.array(vec_b)
6     return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
7
8 # Example
9 sim = cosine_similarity(
10     embed("user prefers Python for backend work"),
11     embed("which programming language does the user like?")
12 )
13 # sim ~= 0.91 --> very similar meaning, returned as a match
```

Listing 6.1: Computing cosine similarity in Python

6.4 Why This Matters — Keyword vs. Semantic Search

Query	Keyword Search	Semantic Search
“coding style?”	Finds only text with “coding style”	Also finds “programming preference”, “language choice”, “code aesthetic”
“deploy process”	Finds only “deploy process”	Also finds “release workflow”, “shipping steps”, “go-live procedure”

Chapter 7

Memory Decay and Confidence Scoring

7.1 Why Memories Need to Decay

Not all memories remain equally valid over time. A memory stating “*user works at Company X*” becomes less reliable after two years. A memory reinforced independently ten times is more trustworthy than one observed only once. The confidence model handles both cases.

7.2 The Confidence Score Formula

Each memory has a confidence score C computed as:

$$C(t) = C_{\max}(R) \cdot e^{-t/\tau(R)} \quad (7.1)$$

where:

$$C_{\max}(R) = 1 - e^{-0.15 R} \quad (\text{initial confidence, grows with reinforcement count } R) \quad (7.2)$$

$$\tau(R) = 90 \cdot \ln(R + 1) \quad (\text{decay half-life in days, extends with } R) \quad (7.3)$$

```
1 import math, time
2
3 def confidence_score(initial: float, written_at: float,
4                     reinforcements: int) -> float:
5     days_elapsed = (time.time() - written_at) / 86400
6     C_max = 1.0 - math.exp(-0.15 * reinforcements)
7     tau = 90.0 * math.log(reinforcements + 1 + 1e-9)
8     if tau < 1:
9         return 0.0
10    return C_max * math.exp(-days_elapsed / tau)
```

Listing 7.1: Confidence score implementation

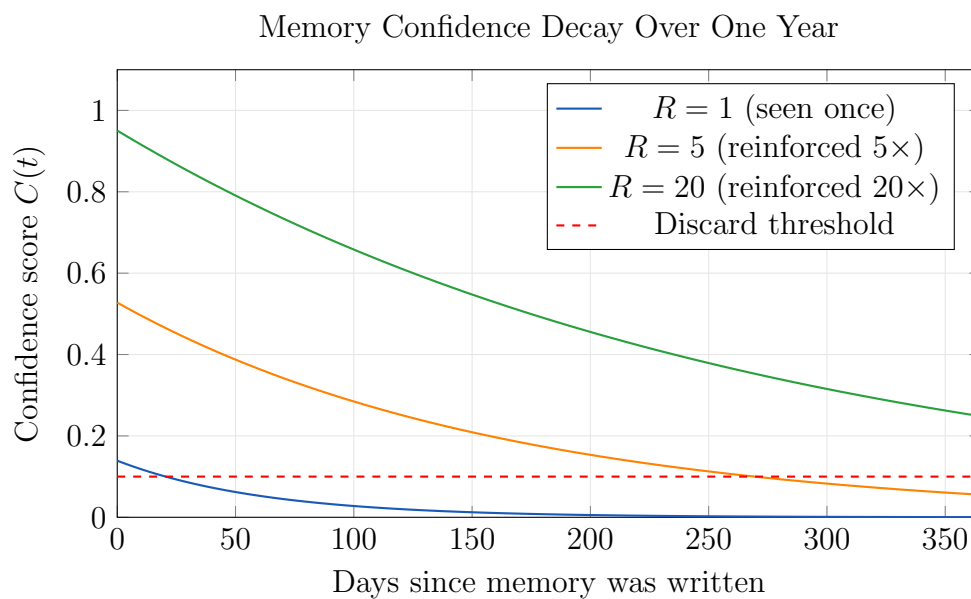


Figure 7.1: Memories reinforced more times start higher and decay more slowly

7.3 Decay Curve

Key Insight

A memory reinforced 20 times stays above the discard threshold for the entire year, while a single-observation memory fades within weeks. This mirrors human memory — frequent use keeps knowledge alive.

Chapter 8

API Reference

All endpoints are served on `http://localhost:6400`. All request and response bodies are JSON. Interactive documentation is auto-generated at `/docs` by FastAPI.

8.1 Health Check

```
1 GET /ping
2 # Response: { "status": "ok", "uptime_seconds": 3600 }
```

8.2 Write a Memory

```
1 POST /memory/write
2 {
3   "type":      "semantic",      # episodic / semantic / procedural /
   working
4   "content":   "User prefers Python",
5   "source":    "my-app",        # which application is writing
6   "confidence": 0.9,            # 0.0 to 1.0
7   "ttl":       null,           # seconds; only used for type=working
8   "tags":      ["preference", "language"]
9 }
10 # Response
11 { "status": "ok", "id": "mem_a1b2c3", "type": "semantic" }
```

Listing 8.1: POST `/memory/write`

8.3 Recall by Semantic Search

```
1 GET /memory/recall?q=what+language+does+the+user+prefer&type=semantic&
   top_k=5
2
3 # Response
4 {
5   "results": [
6     {
7       "id":      "mem_a1b2c3",
```

```
8      "content":      "User prefers Python over JavaScript",
9      "confidence": 0.87,
10     "similarity": 0.94,
11     "written_at": "2026-05-15T10:30:00"
12   }
13 ]
14 }
```

Listing 8.2: GET /memory/recall – semantic

8.4 Recall by Time Range (Episodic)

```
1 GET /memory/recall?type=episodic&since=2026-05-19&until=2026-05-21
2
3 # Returns: list of events sorted by timestamp within the window
```

Listing 8.3: GET /memory/recall – episodic time range

8.5 Working Memory Operations

```
1 # Get all active working memory keys
2 GET /memory/context
3
4 # Set a working memory key with TTL
5 POST /memory/write
6 { "type": "working", "key": "current_task",
7   "content": "Building cache server", "ttl": 14400 }
8
9 # Response example from GET /memory/context
10 {
11   "current_task": "Building cache server",
12   "active_file": "cache_server.py",
13   "active_branch": "feature/cache-ttl"
14 }
```

Listing 8.4: Working memory endpoints

8.6 Full Endpoint Reference

Endpoint	Method	Description
/ping	GET	Health check
/memory/write	POST	Write any memory type
/memory/recall	GET	Semantic or time-based recall
/memory/context	GET	All working memory keys
/memory/forget	DELETE	Remove a memory by ID
/memory/reinforce	POST	Increment reinforcement count
/memory/associate	POST	Link two memories with a relation
/memory/summary	GET	LLM-generated topic summary
/memory/export	GET	Export all memories to JSON
/memory/import	POST	Import memories from JSON file
/memory/flush	POST	Clear working memory only
/info	GET	Server stats: keys, hit rate, uptime

Chapter 9

Workflow Diagrams

9.1 Write Flow

9.2 Recall Flow

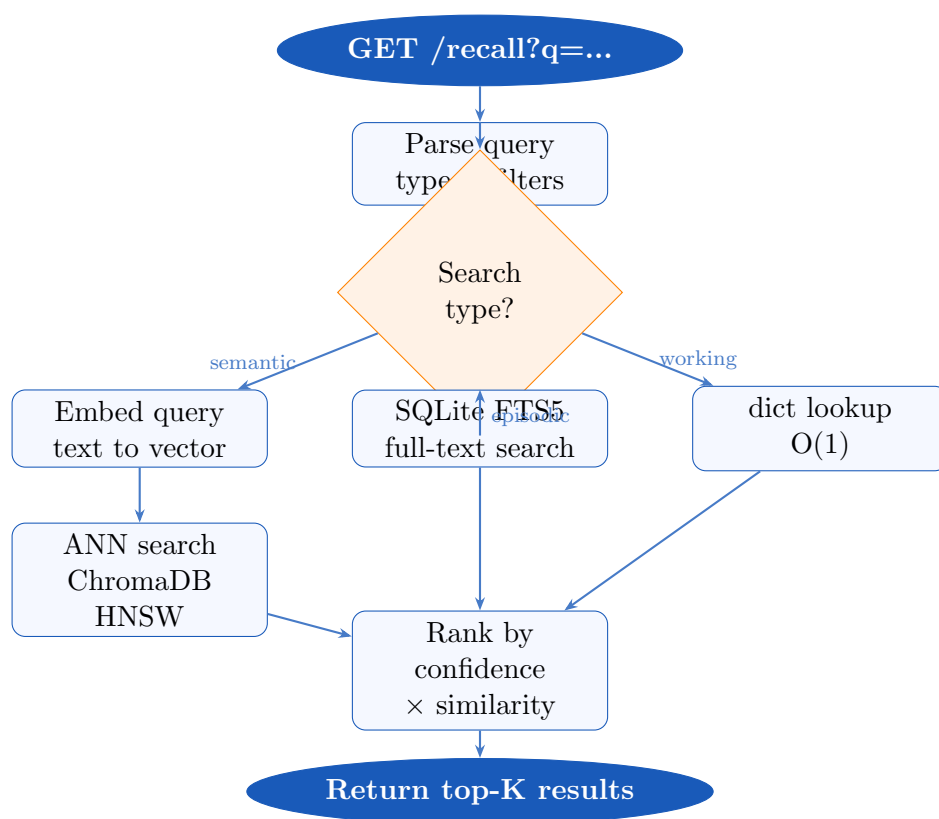


Figure 9.1: Recall flow: different retrieval paths depending on memory type

9.3 Cross-Application Memory Sharing

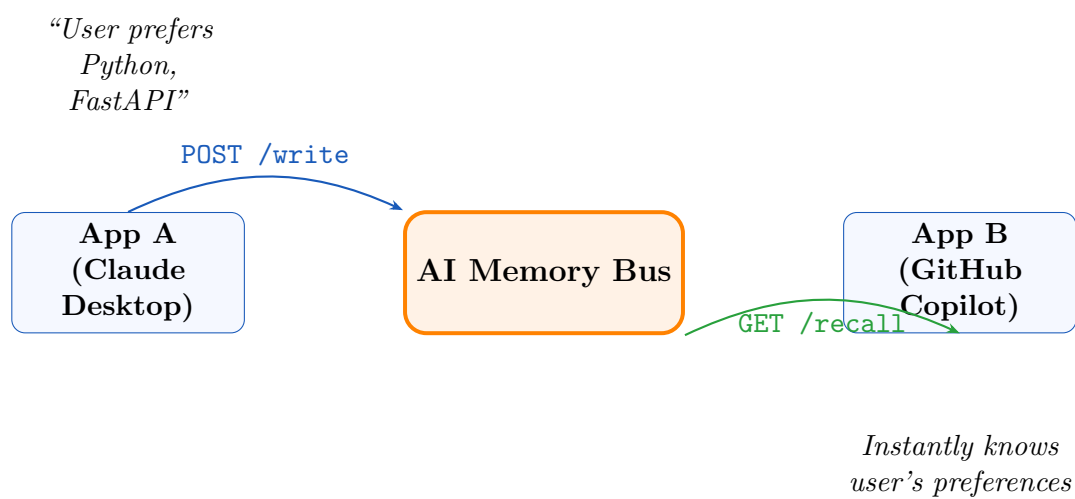


Figure 9.2: What App A writes, App B immediately knows

Chapter 10

Use Cases

10.1 Use Case 1: Instant Onboarding for New AI Tools

Scenario: A new AI tool is installed. It queries the Memory Bus on first launch and immediately knows everything about the user — no manual configuration required.

```
1 import requests
2
3 def load_user_context() -> list:
4     r = requests.get("http://localhost:6400/memory/recall", params={
5         "type": "semantic",
6         "q": "user preferences stack company role",
7         "top_k": 20
8     })
9     return r.json()["results"]
10 # Returns: preferred language, stack, company, coding style, team
    context
11 # Zero manual setup needed
```

Listing 10.1: New tool bootstraps context from Memory Bus

10.2 Use Case 2: Job Deduplication

Scenario: A background worker uses working memory to ensure the same job is never processed twice.

```
1 def process_job(job_id: str):
2     ctx = requests.get("http://localhost:6400/memory/context").json()
3     if f"job:{job_id}" in ctx:
4         return "already_processed"
5
6     do_the_work(job_id)
7
8     requests.post("http://localhost:6400/memory/write", json={
9         "type": "working",
10        "key": f"job:{job_id}",
11        "content": "done",
12        "ttl": 86400 # forget after 24 hours
13    })
```

Listing 10.2: Deduplication with working memory TTL

10.3 Use Case 3: Cross-Tool Active Context

Scenario: A VS Code plugin writes the current file and task; Claude Desktop reads it without being told.

```
1 POST /memory/write
2 {
3   "type":      "working",
4   "key":       "active_file",
5   "content":   "src/auth/token_validator.py",
6   "ttl":       1800,
7   "source":    "vscode-plugin"
8 }
```

Listing 10.3: VS Code plugin writes context on each file save

10.4 Use Case 4: Developer History Recall

Scenario: A developer asks their AI assistant: “What was I working on last Tuesday?”

```
1 GET /memory/recall?type=episodic&since=2026-05-19&until=2026-05-20
2
3 # Returns timestamped activity log for that day:
4 # - 09:00 Fixed CORS config in API gateway
5 # - 11:30 Added integration tests for payment module
6 # - 14:00 Code review for auth PR
```

Listing 10.4: Episodic recall by date range

10.5 Use Case Summary

Use Case	Memory Type	Benefit
New tool onboarding	Semantic	Zero config, instant full context
Job deduplication	Working (TTL)	Never process the same job twice
Context sharing	Working	Cross-tool session continuity
Debugging history	Episodic	Recall exactly what you worked on
Team knowledge base	Semantic+Episodic	New devs onboard in minutes
Workflow automation	Procedural	AI suggests correct deploy steps
Preference recall	Semantic	Never repeat your preferences

Chapter 11

Technology Stack

Layer	Technology	Why Chosen	Alternative
Language	Python 3.11+	Readable, rich ecosystem	Go (faster raw throughput)
HTTP Server	FastAPI	Async, typed, auto-docs	Flask (simpler, sync)
Episodic DB	SQLite + FTS5	Zero setup, stdlib	PostgreSQL (overkill)
Vector DB	ChromaDB	Local-only, no extra server	FAISS (lower-level)
Embeddings	sentence-transformers	Offline, fast, accurate	OpenAI API (needs internet)
Math	NumPy	Fast vector operations	Pure Python (too slow)
Working Mem	dict + heapq	No deps, pure RAM	Redis (external process)
Windows Svc	NSSM	Run at boot, services.msc	Task Scheduler

11.1 Dependencies

```
1 fastapi>=0.110.0
2 uvicorn>=0.29.0
3 chromadb>=0.4.0
4 sentence-transformers>=2.7.0
5 numpy>=1.26.0
6 pydantic>=2.6.0
7 # sqlite3 is part of Python stdlib -- no install needed
```

Listing 11.1: requirements.txt

```
1 pip install fastapi uvicorn chromadb sentence-transformers numpy
   pydantic
```

Listing 11.2: Install command

Chapter 12

Implementation Roadmap

12.1 Phase-by-Phase Plan

Phase	Name	Deliverables
1	Core Data Structures	<code>MemoryStore</code> class: working memory with dict, heapq, and OrderedDict (LRU)
2	Persistent Storage	SQLite schema for episodic & procedural; ChromaDB initialisation
3	Embeddings	Integrate sentence-transformers; cosine similarity recall
4	HTTP API	FastAPI server with all endpoints; Swagger UI at <code>/docs</code>
5	Confidence Decay	Implement scoring formula; background decay cleanup task
6	Client Library	<code>memory_client.py</code> importable by any Python service
7	Windows Service	NSSM install; auto-start at boot; visible in <code>services.msc</code>
8	Monitoring Dashboard	<code>/info</code> endpoint; terminal dashboard with <code>rich</code> library

12.2 Estimated Timeline

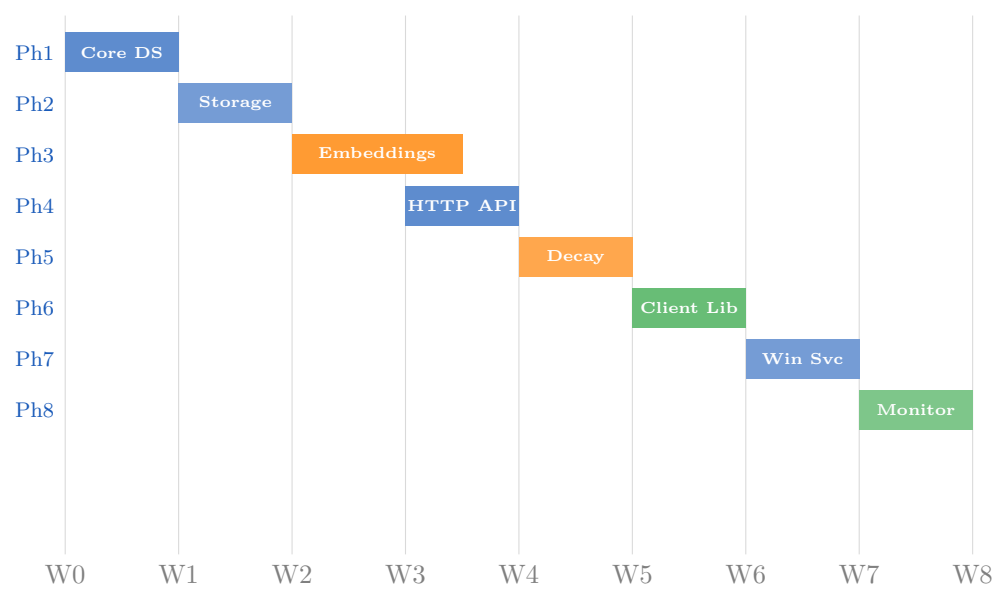


Figure 12.1: 8-week implementation timeline (W = week)

Chapter 13

Future Scope

1. **Memory Graph** — Link memories with typed relationships (*caused_by*, *related_to*, *contradicts*). Navigate the graph for multi-hop context retrieval.
2. **Real-Time Pub/Sub** — When a new memory is written, notify subscribed applications in real time via WebSocket or SSE.
3. **Team Sync** — Optionally sync selected non-private memories across team members over a local network.
4. **LLM Summarisation** — Use a local LLM (Ollama) to generate natural-language summaries of everything known about a topic.
5. **Conflict Resolution** — When two memories contradict each other, automatically flag and resolve based on recency and confidence.
6. **Plugin API** — Allow third-party apps to register custom memory types and recall strategies.
7. **Encryption at Rest** — Encrypt the SQLite and ChromaDB stores with a user-provided passphrase.
8. **Cross-Machine Export** — Full memory export as a portable JSON bundle; import into a new machine or share with a team.

Chapter 14

Conclusion

The AI Memory Bus addresses a genuine gap in today's AI tooling ecosystem. While individual AI applications grow increasingly capable, they remain isolated and amnesiac by default — each starting from zero every session, each storing context in a proprietary format the others cannot access.

By implementing a shared local memory layer backed by proven data structures (HashMap, OrderedDict, Min-Heap, HNSW graph) and modern NLP (vector embeddings, cosine similarity), every AI tool on the machine gains a common, persistent, semantically-searchable intelligence layer.

The four-tier memory model mirrors human cognition deliberately. Different kinds of knowledge have different optimal storage and retrieval strategies; conflating them into a single flat store loses information. The confidence decay model ensures memories remain reliable over time without manual curation.

Key Takeaways for New Users

1. The AI Memory Bus is a local server — always running, nothing in the cloud.
2. Any app calls it via simple HTTP; no special SDK required.
3. Four memory types handle four kinds of knowledge, each stored differently.
4. Semantic search finds *meaning*, not just matching words.
5. Confidence scoring makes old or contradicted memories fade naturally.
6. The underlying data structures — HashMap, Min-Heap, HNSW — are the same building blocks used in production databases and caches.

End of Document
